```
  File "/.../pathlib.py", line 1056, in read_text
    with self.open(mode='r', encoding=encoding, errors=errors) as f:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/.../pathlib.py", line 1042, in open
    return io.open(self, mode, buffering, encoding, errors, newline)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
❸ FileNotFoundError: [Errno 2] No such file or directory: 'alice.txt'
```

This is a longer traceback than the ones we've seen previously, so let's look at how you can make sense of more complex tracebacks. It's often best to start at the very end of the traceback. On the last line, we can see that a `FileNotFoundError` exception was raised ❸. This is important because it tells us what kind of exception to use in the `except` block that we'll write.

Looking back near the beginning of the traceback ❶, we can see that the error occurred at line 4 in the file *alice.py*. The next line shows the line of code that caused the error ❷. The rest of the traceback shows some code from the libraries that are involved in opening and reading from files. You don't usually need to read through or understand all of these lines in a traceback.

To handle the error that's being raised, the `try` block will begin with the line that was identified as problematic in the traceback. In our example, this is the line that contains `read_text()`:

```
from pathlib import Path

path = Path('alice.txt')
try:
    contents = path.read_text(encoding='utf-8')
❶ except FileNotFoundError:
    print(f"Sorry, the file {path} does not exist.")
```

In this example, the code in the `try` block produces a `FileNotFoundError`, so we write an `except` block that matches that error ❶. Python then runs the code in that block when the file can't be found, and the result is a friendly error message instead of a traceback:

```
Sorry, the file alice.txt does not exist.
```

The program has nothing more to do if the file doesn't exist, so this is all the output we see. Let's build on this example and see how exception handling can help when you're working with more than one file.

Analyzing Text

You can analyze text files containing entire books. Many classic works of literature are available as simple text files because they are in the public domain. The texts used in this section come from Project Gutenberg (*https://gutenberg.org*). Project Gutenberg maintains a collection of literary works that are available in the public domain, and it's a great resource if you're interested in working with literary texts in your programming projects.